

**INTERNATIONAL CYBERSECURITY
AND DIGITAL FORENSICS ACADEMY**

**INTRO TO CRITICAL INFRASTRUCTURE
FOR IT LEARNERS**

LAB 06 — INTRODUCTION TO MODBUS

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 06: Introduction to Modbus

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 3–4 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Explain the difference between Modbus TCP and Modbus RTU.
- Identify common Modbus function codes and data areas.
- Observe Modbus TCP communication between FUXA and OpenPLC in an isolated lab.
- Use Wireshark to capture/filter Modbus traffic and identify request/response fields.
- Relate register/address information to the running tank-fill process.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password
SCADA VM	192.168.2.10	scada	password
Analyst/Kali VM	Instructor-provided	kali	kali

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Reference: Modbus Function Codes

Code	Function	Purpose
01	Read Coils	Read discrete output/coils
02	Read Discrete Inputs	Read discrete input states
03	Read Holding Registers	Read 16-bit holding registers
04	Read Input Registers	Read 16-bit input registers
05	Write Single Coil	Change one discrete coil
06	Write Single Holding Register	Change one holding register
15	Write Multiple Coils	Change multiple coils
16	Write Multiple Holding Registers	Change multiple registers

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Note: Due to issues with FUXA malfunctioning (not loading JSON file & Modbus not available), the first part of the LAB will not be completed

Step 1. Start the PLC, SCADA and analyst VMs on the instructor-provided isolated laboratory topology. Do not connect this lab to production or bridged networks.

Step 2. On the PLC VM, start the TankFillSim program in OpenPLC Runtime and confirm Running status.

Step 3. On the SCADA VM, start FUXA, load the tank-fill HMI and confirm the openplc connection targets 192.168.1.10:502.

Step 4. Run the HMI and verify that the tank level changes between its operating limits.

Step 5. On the analyst VM, open Wireshark. Select only the interface connected to the ICDFA lab segment carrying PLC/SCADA traffic.

Step 6. Apply the display filter modbus. If needed, refine it with tcp.port == 502.

Step 7. Start a short packet capture while the HMI is running, allow several process updates to occur, then stop the capture.

Step 8. Select a Modbus request/response pair and expand the Modbus/TCP and Modbus protocol fields.

Step 9. Identify the transaction/request, source and destination IP addresses, function code and referenced register/data value.

Step 10. Locate a Function Code 4 (Read Input Registers) response associated with the tank-level value. In the teaching image, Input Register 80 corresponds to the Tank Level indication.

Step 11. Complete the Modbus function-code table by explaining at least codes 01, 02, 03, 04, 05, 06, 15 and 16.

Step 12. Save the packet capture with a filename containing your registration number and LAB06. Do not capture unrelated traffic.

Required Evidence & Submission

- Screenshot of the FUXA/OpenPLC process running.
- Screenshot of Wireshark filtered to Modbus traffic.
- One annotated request/response analysis identifying function code and register.
- The saved .pcap/.pcapng capture from the isolated training network.
- A completed function-code table and a short comparison of Modbus TCP versus Modbus RTU.

Recommended filename: <RegistrationNo>_LAB06_Introduction_to_Modbus.pdf plus the required project/capture file(s).

Function-code table and a short comparison of Modbus TCP versus Modbus RTU

This table below covers the most common standard function codes used to interact with a device's memory mapping:

Code (Decimal)	Function Name	Data Type	Access Type	Primary Purpose
FC01	Read coil	1 bit (Digital)	Read-Write	Reads the ON/OFF status of discrete outputs (e.g., relays).
FC02	Read Discrete Inputs	1 bit (Digital)	Read-Only	Reads the ON/OFF status of physical inputs (e.g., limit switches).
FC03	Read Holding Registers	16-bit (Analog)	Read-Write	Reads internal numerical configuration values or setpoints.
FC04	Read Input Registers	16-bit (Analog)	Read-Only	Reads numerical measurements from physical variables (e.g., temperature).
FC05	Write Single Coil	1 bit (Digital)	Write-Only	Changes the ON/OFF state of a single discrete output.
FC06	Write Single Register	16-bit (Analog)	Write-Only	Overwrites a single numerical value in a configuration register.
FC15 (0x0F)	Write Multiple Coils	1 bit (Digital)	Write-Only	Changes the state of multiple discrete outputs simultaneously.
FC16 (0x10)	Write Multiple Registers	16-bit (Analog)	Write-Only	Overwrites numerical values across a block of consecutive registers.

Knowledge Check

1. What transport/networking difference separates Modbus TCP from Modbus RTU?

Response: The primary difference that separates Modbus TCP from Modbus RTU is the underlying transport and network layers used to transmit data, as both protocols share an identical application layer (the same function codes and register structures). Modbus RTU is a serial protocol, meaning it relies on sequential data streams usually running over RS-485 or RS-232 physical lines. Modbus TCP is an Ethernet-based protocol, which encapsulates Modbus data inside standard TCP/IP packets transmitted over local or wide-area networks.

2. Why does Modbus use function codes?

Response: Modbus uses function codes as the "verbs" of its communication protocol to tell a receiving device exactly what action to perform and which memory table to access. Because Modbus is a simple, lightweight protocol, it relies on a single 1-byte identifier (the function code) inside the message frame to define both the operation type (read or write) and the data type (1-bit coils or 16-bit registers).

3. What does Function Code 4 request/read?

Response: Modbus Function Code 4 (FC04) requests and reads the values of 16-bit Input Registers, which are read-only numerical data points representing physical measurements, sensor values, or device status metrics. Because these registers are read-only, a Modbus master can request data from them to monitor a system, but it cannot alter or write new values to them.

4. Why is visibility of unprotected industrial protocol traffic a cybersecurity concern?

Response: The visibility of unprotected industrial protocol traffic is a critical cybersecurity concern because it allows attackers to easily intercept operational data, map out physical infrastructure, and inject malicious commands without any authentication. Because classic operational technology (OT) protocols like standard Modbus or EtherNet/IP were designed decades ago for isolated networks, they lack encryption, digital signatures, and user access controls.

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.